

Kombinovano

Nedelja 1: Uvod u kriptografiju

Uvod

Kriptografija je nauka koja nam omogućava da osiguramo tajnost, integritet i autenticnost pri komunikaciji. Sprečava neovlašćeno citanje i promene podataka i omogućava nam da znamo identitet autora.

1. **Tajnost (Confidentiality)** – niko neovlašćen ne može da pročitati poruku
2. **Integritet (Integrity)** – poruka nije menjana tokom prenosa
3. **Autentičnost (Authenticity)** – znamo ko je autor poruke

Neki osnovni pojmovi koji će nam trebati da bi dalje mogli da pratimo kurs (ostali će biti uvedeni usputno):

- **OT (Otvoreni tekst)** je poruka koji treba poslati npr. "ZDRAVO"
- **ST (Šifrat)** je sifrovana poruka npr. "XQABER"
- **Sifrovanje** je transformacija otvorenog teksta u šifrat - $C = E(K, M)$, gde je M poruka, K ključ, a C šifrat
- **Desifrovanje** je inverzna operacija sifrovanju, transformise šifrat u otvoreni tekst, vazi $M = D(K, C)$ i $D(K, E(K, M)) = M$
- **Kodiranje** transformise otvoreni tekst u niz cifara ili bita. Npr. velika slova abecede mogu da se kodiraju sa A -> 0, B -> 1, ..., Z -> 25 (ZDRAVO -> 25 3 17 0 21 14) ili npr. ASCII kod (A -> 01000001, B -> 01000010, ...)
- **Dekodiranje** je obrnuta transformacija kodiranju, transformise niz cifara ili bita u polazni tekst.

Vecina bezbednosti u kriptografiji se oslanja na svojstva velikih brojeva. Čak i ako postoje (relativno) efikasni algoritmi za razbijanje vecine bezbednosih sistema oni u praksi nisu izvoljivi zbog same velicine brojeva u pitanju. Pogledajmo par primera da steknemo osećaj zašto su neki napadi nepraktični:

- Broj kombinacija za loto (6 od 45): oko 8 miliona $\approx 2^{23}$
- Rastojanje Beograd-Atina u milimetrima: $\sim 600 \text{ km} = 6 \times 10^8 \approx 2^{30}$
- Molekuli vazduha u prostoriji: $\sim 10^{27} \approx 2^{90}$
- AES-128 ključ: $2^{128} \approx 3.4 \times 10^{38}$ mogućih ključeva
- Računar koji proba 10^9 ključeva/sec treba $\sim 10^{22}$ godina za brute-force AES-128

Poenta: ako je prostor ključeva dovoljno veliki ($\geq 2^{128}$), brute-force napad je nemoguć sa trenutnom tehnologijom.

Cezarova i Viženerova sifra

Ljudi su još davno uvideli potrebu za sifrovanjem (motivisani čuvanjem i prenošenjem tajni u ratu). Jedan od najpoznatijih primera sifrovanja je **Cezarova sifra**. Ona je sifra zamene, koja svako slovo zamenjuje trećim slovom udesno (duž abecede) od njega. Ako se radi o engleskoj abecedi onda A → D, B → E, ..., Z → C.

Cezarova šifra

Najjednostavniji šifarski sistem – svako slovo se pomera za fiksnu vrednost (ključ).

Kako radi:

- Alfabet: A=0, B=1, C=2, ..., Z=25
- Šifrovanje: $c = (m + k) \bmod 26$
- Dešifrovanje: $m = (c - k) \bmod 26$

Primer (ključ k=3):

Otvoreni: A B C D E F ... X Y Z

Šifrovani: D E F G H I ... A B C

"NAPAD" → "QDSDG"

Prostor ključeva: samo 26 mogućih ključeva → trivijalno se razbija (probaj svih 26).

Kriptoanaliza Cezarove šifre:

- Brute force:** Probaj svih 26 ključeva, pogledaj koji daje smisleni tekst.
- Frekventna analiza:** U engleskom jeziku slovo 'E' je najčešće (~12.7%). Nađi najčešće slovo u šifratu → pomeranje od 'E' do tog slova je verovatno ključ.

Vizenerova sifra

Ovo se može uopstiti. Ako je moguće pomeranje u abecedi za proizvoljan broj pozicija tada se radi o **monoalfabetskoj cifri translacije** i tada je Cezarova sifra samo specijalni slučaj ovog sistema.

Najpoznatiji primer ovog sistema je **Viznerova sifra (Vigener)**.

Vigener je proširenje Cezarove šifre – koristi se ključ-reč umesto jednog broja. Svako slovo poruke se šifrira drugim pomeranjem.

Kako radi:

- Ključ: reč koja se ciklično ponavlja
- Šifrovanje: $c_i = (m_i + k_i \bmod \text{len}(\text{key})) \bmod 26$
- Dešifrovanje: $m_i = (c_i - k_i \bmod \text{len}(\text{key})) \bmod 26$

Primer (ključ = "KEY"):

Otvoreni tekst: N A P A D
Ključ (ciklus): K E Y K E
Pomeranja: 10 4 24 10 4
Šifrovani tekst: X E N K H

Prednost nad Cezarovom: isto slovo u otvorenom tekstu se šifruje u različita slova (npr. oba 'A' daju različite rezultate jer se koriste različita pomeranja).

Slabost: Ključ se ponavlja ciklično. Ako napadač otkrije dužinu ključa (npr. Kasiski test ili indeks koincidencije), problem se svodi na više nezavisnih Cezarovih šifri.

One-Time Pad

Jednokratna sifra (One-time pad) je sifra koja je teorijski neprobojna ako se koristi na pravilan način. Ključ je niz bitova koji je jednako dug kao i poruka. Enkripcija se vrši tako što se poruka kombinuje sa ključem pomoću XOR-a.

$E(k, m) = k \text{ XOR } m$, $D(k, c) = k \text{ XOR } c$.

Kako bi sifra zaista bila neprobojna, ključ mora biti slučajno generisan, iste dužine kao i poruka, koriscen samo jednom i cuvan u tajnosti. Ako bar jedan od ovih uslova nije ispunjen, sifra postaje podložna napadima

Afina šifra

Afina šifra je vrsta **monoalfabetske zamene** koja uvodi dodatni matematički korak. Svako slovo se prvo preslikava u numeričku vrednost (A=0, B=1, ..., Z=25), a zatim se transformiše linearnom jednačinom:

- Šifrovanje: $e(x) = (a \cdot x + b) \bmod 26$
- Dešifrovanje: $d(y) = a^{-1} \cdot (y - b) \bmod 26$

gde su **a** i **b** ključevi šifre. **Bitno:** **a** mora biti uzajamno prosto sa 26 (tj. $\text{NZD}(a, 26) = 1$), inače inverz ne postoji i dešifrovanje nije moguće.

Primer (a=5, b=8): slovo H (x=7) $\rightarrow e(7) = (5 \cdot 7 + 8) \bmod 26 = 43 \bmod 26 = 17 \rightarrow R$

Cezarova šifra je specijalni slučaj afine šifre gde je a=1. Prostor ključeva je veći od Cezarove (12 mogućih vrednosti za $a \times 26$ za b = 312), ali se i dalje lako razbija frekventnom analizom jer je u pitanju monoalfabetska zamena.

Plejferova šifra (bigrami)

Plejferova šifra ne šifruje pojedinačna slova, već **parove slova (bigrame)**. Koristi matricu 5×5 popunjenu slovima na osnovu ključne reči. Koristila se u Prvom svetskom ratu.

Kreiranje matrice: Upiši jedinstvena slova ključne reči u matricu (bez ponavljanja), pa popuni ostatak preostalim slovima abecede. Slova I i J dele jedno polje.

Priprema teksta: Tekst se deli u parove. Ako su oba slova u paru ista (npr. LL), ubacuje se X između → LX, L... Neparan broj slova – dodaj X na kraj.

Pravila šifrovanja:

1. **Isti red:** svako slovo se zameni onim desno od njega (kružno)
2. **Ista kolona:** svako slovo se zameni onim ispod njega (kružno)
3. **Pravougaonik:** svako slovo se zameni onim u istom redu ali u koloni drugog slova

Prednost: Razbija jednostavnu frekventnu analizu pojedinačnih slova jer se bigram šifruje kao celina.
Slabost: i dalje podložna analizi frekvencija bigrama.

Kerckhoffsov princip

Bezbednost šifarskog sistema mora da zavisi **isključivo od tajnosti ključa**, a ne od tajnosti algoritma.

Ovo znači: pretpostavlja se da napadač zna potpuno kako algoritam radi. Jedina tajna je ključ. Svi moderni kriptografski sistemi poštuju ovaj princip – AES, RSA, itd. su javno poznati algoritmi.

Kriptoanaliza

Kriptoanaliza je proces pomoću kojeg napadač pokušava da šifrat transformiše u otvoreni tekst, ne znajući ključ.

U zavisnosti od toga kojim informacijama raspolaže napadač, napade delimo u tri vrste:

1. **Napad sa poznavanjem samo šifrata** – Napadač ima samo presretnutu šifrovanu poruku. Oslanja se na statističku analizu (frekventna analiza slova, bigrama, trigrama) ili brute-force. Kod monoalfabetske zamene, ne pojavljuju se sva slova podjednako često (u srpskom 'a' je najčešće, u engleskom 'e'), što olakšava napad.
2. **Napad sa poznatim otvorenim tekstom** – Napadač ima neke parove (otvoreni tekst, šifrat). Na primer, zna da svaka poruka počinje sa "Poštovani" ili da mrežni paketi imaju standardno zaglavlje. Cilj je iz tih parova dedukovati ključ.
3. **Napad sa izabranim otvorenim tekstom** – Napadač može da izabere proizvoljni tekst, ubaci ga u sistem za šifrovanje i dobije rezultat. Šalje matematički pažljivo birane poruke da otkrije strukturu ključa. Realan kod pametnih kartica, bankomata, veb servisa.

Simetricni i Asimetricni kriptografski sistemi

Simetricna kriptografija, blokvske i protocne sifre

Simetrična kriptografija (poznata i kao kriptografija sa privatnim ključem) predstavlja najstariji i najbrži oblik enkripcije. Njena glavna karakteristika je da se isti ključ koristi i za zaključavanje (enkripciju) i za otključavanje (dekripciju) podataka.

Formalnije receno, ako je P originalni text (plaintext), K tajni ključ, E funkcija enkripcije tada se šifrovan tekst C dobija kao: $C = E(K, P)$, Dekripcija se vrši primenom inverzne funkcije D uz pomoć istog ključa: $P = D(K, C)$.

Simetrični sistemi se dele na dve glavne kategorije u zavisnosti od toga kako obrađuju podatke:

- **Blok šifre (Block Ciphers)** - Podaci se dele na blokove fiksne veličine (npr. 64 ili 128 bita). Svaki blok se šifrjuje zasebno.
 - **AES (Advanced Encryption Standard):** Trenutni standard. Koristi blokove od 128 bita i ključeve od 128, 192 ili 256 bita. Izuzetno je siguran i hardverski optimizovan.
 - **DES i 3DES:** Stariji standardi. DES je danas nesiguran zbog kratkog ključa (56 bita), dok se 3DES polako povlači iz upotrebe.
- **Protočne šifre (Stream Ciphers)** - Podaci se šifruju bit po bit ili bajt po bajt u neprekidnom nizu. Obično su brže od blok šifri i koriste se tamo gde je bitno nisko kasnjenje.
 - **ChaCha20:** Moderan i veoma brz algoritam, često se koristi u mobilnim uređajima i TLS protokolu.
 - **RC4:** Nekada popularan, ali danas se smatra nesigurnim.

Prednosti	Mane
Brzina: Ekstremno brzi algoritmi, pogodni za velike količine podataka.	Distribucija ključa: Najveći problem - kako bezbedno poslati ključ drugoj strani a da ga niko ne presretne?
Mala procesorska snaga: Idealni za IoT uređaje i mobilne telefone.	Skalabilnost: Ako 100 ljudi treba da komunicira međusobno, svakom paru je potreban unikatan ključ, što vodi do ogromnog broja ključeva - $n(n-1)/2$.
Efikasnost: Šifrovani podaci obično ne zauzimaju više prostora od originalnih.	Nema neporecivosti: Pošto obe strane imaju isti ključ, ne može se dokazati ko je tačno kreirao poruku.

Protočne i blokvske šifre

Protočne šifre šifruju podatke **bit po bit** ili bajt po bajt. One kombinuju izvorni tekst sa neprekidnim nizom bitova koji se naziva **keystream**. **Princip rada:** Koristi se logička operacija **XOR** ($\oplus$). Ako je P bit teksta, a K bit ključa, šifrat je $C = P \oplus K$. Prednost ovog pristupa je velika brzina i malo kasnjenje (za razliku od blok šifre ovde nema čekanja da se sledeći blok obradi). Problem je činjenica da se ključni niz nikada ne sme ponoviti sa istim ključem (u suportnom postoji opasnost od Two-Time Pad napada).

Pošto je nemoguće razmeniti beskonačno dugačak nasumični ključ, koristimo **PRNG (Pseudo Random Number Generator)**. On uzima kratak, tajni ključ koji se naziva seme (**seed**) i pomoću matematičkog algoritma generise ogroman, naizgled nasumican niz bitova (**keystream**).

Blokovski sistemi obrađuju fiksne grupe bita (blokove) koristeći složene matematičke transformacije unutar konačnih polja. Imamo dve vrste standarda blokovskog šifrovanja:

- **DES (Data Encryption Standard)** - Klasična blokovska šifra sa blokom od 64 bita i ključem od 56 bita. Koristi Feistelovu mrežu, ali se danas smatra prevaziđenom zbog male dužine ključa.
- **AES (Advanced Encryption Standard)** - Savremeni standard, postoje četiri uobičajena načina rada AES-a. Postoje AES šifrue blokove podataka od 128 bita, ovi modovi definišu kako ćemo šifrovati dugacku poruku koja se sastoji od mnogo takvih blokova:
 - Najjednostavniji i najnesigurniji način rada je **ECB (electronic code book)**. Svaki blok otvorenog teksta se šifrue potpuno nezavisno od ostalih, koristeći isti ključ. Problem sa ovim pristupom je ako u poruci imamo dva identična bloka podataka, dobijamo dva identična bloka šifrata - ovo otkriva obrasce u podacima.
 - Drugi i najčešći način rada je **CBC (cipher block chaining)**. Za razliku od ECB-a ovde se uzodi zavisnost između blokova kako bi se sakrili podaci. Pre nego što se blok otvorenog teksta šifrue, on se XOR-uje sa šifratom prethodnog bloka. Za prvi blok se koristi poseban, unapred odabran, ne-tajni broj koji se naziva **IV (inicijalni vektor)**. Stavši IV se šalje pre svake poruke.

Asimetrična kriptografija

U simetričnim sistemima (poput AES-a), pošiljalac i primalac moraju imati isti tajni ključ. Ako taj ključ neko presretne tokom razmene, cela komunikacija je kompromitovana. **Asimetrična kriptografija** ovo rešava korišćenjem para ključeva.

Svaki korisnik u sistemu generise par matematički povezanih ključeva:

- **Javni ključ (Public Key)**: Može se slobodno deliti sa bilo kim. Služi za šifrovanje podataka ili proveru digitalnog potpisa.
- **Privatni ključ (Private Key)**: Čuva se u strogoj tajnosti. Služi za dešifrovanje podataka ili kreiranje digitalnog potpisa.

Ključevi funkcionisu po principu - ono što jedan ključ zaključa, samo onaj drugi iz istog para može da otključa. Nemoguće je u razumnom vremenu izračunati privatni ključ na osnovu poznavanja javnog ključa.

Ako osoba A (pošiljalac) želi da pošalje tajnu poruku osobi B (primalac):

1. Osoba A uzima javni ključ osobe B.
2. Osoba A šifrue poruku tim javnim ključem.
3. Sada tu poruku može da dešifrue isključivo osoba B, jer samo ona poseduje odgovarajući privatni ključ.

Najpoznatiji primeri su RSA, ECC i Diffie-Hellmann. Najveća prednost asimetričnog šifrovanja je to što nema potrebe za razmenom poverljivih ključeva. Ovo dolazi po cenu brzine jer su ovi algoritmi mnogo sporiji i često se desava da je sam šifrat veći od poruke. U praksi se često koriste hibridni pristupi, gde se asimetrična kriptografija koristi samo na početku za uspostavljanje tajnog ključa.

Heširanje

Heširanje je proces transformacije ulaznih podataka bilo koje veličine u izlaz fiksne dužine, koji se naziva **heš vrednost** korišćenjem neke funkcije za heširanje. Za razliku od enkripcije, heširanje je **jednosmerna funkcija** tj. dizajniraju se tako da bude nemoguće vratiti heš u originalne podatke. Služe nam da obezbedimo da poruku koju smo primili neko usput nije izmenio, osobina poruke da nije dozvela izmene je **integritet**. Svaka heš funkcija treba da ima sledeća svojstva:

- Potrebno je da se njene vrednosti lako i brzo izracunavaju
- Heš funkcija treba da bude jednosmerna funkcija (za zadato y potrebno je da bude jako tesko ili nemoguće određivanje x takvog da $H(x) = y$)
- Za zadato x potrebno je da bude jako tesko određivanje drugog x' t.d. $H(x') = H(x)$. Ova osobina se naziva **osnovna otpornost na koliziju**

Ako je pored toga ispunjen zahtev da je tesko pronaci bilo koji par x, x' takav da je $H(x) = H(x')$, onda se za H kaže da ima **jaku otpornost za koliziju**.

Da se napravi algoritam za heširanje obično se polazi od funkcije f koja blokove od $m + t$ bita preslikava u blokove od t bita, gde su m i t veliki, a f ima sve tri navedene osobine. Ulaz ovog algoritma je poruka, a izlaz njena heš vrednost.

Funkcija f može se iskoristiti za dobijanje heš funkcije. Pretpostavimo da je poruka razbijena u m -bitne blokove $M_1, M_2, \dots, M_k$. Ako dužina poruke nije deljiva sa m , onda se poslednji blok dopunjuje do dužine m . Zadat je unapred dogovoren blok od t bita (inicijalizovani vektor IV).

Nedelja 2: Difi-Helman, Stepenovanje kvadriranjem, problem diskretnog logaritma, blokovske sifre

Osnove teorije brojeva i konačna polja

Kriptografija se oslanja na aritmetiku u **konačnim poljima**. Osnovna ideja: umesto da radimo sa običnim brojevima, radimo sa brojevima **po modulu** nekog broja.

Modularna aritmetika: $a \bmod n$ je ostatak pri deljenju a sa n . Npr. $17 \bmod 5 = 2$.

Ako je p **prost broj**, tada skup $Z_p = \{0, 1, 2, \dots, p-1\}$ sa operacijama sabiranja i množenja po modulu p čini **konačno polje** F_p . Svaki nenulti element u F_p ima **multiplikativni inverz** (tj. za svako $a \neq 0$ postoji b takvo da $a \cdot b \equiv 1 \bmod p$).

Multiplikativna grupa $F_p^* = F_p \setminus \{0\}$ je **ciklična** – postoji element g (naziva se **generator** ili primitivni koren) takav da su svi elementi grupe stepeni od g : $g^1, g^2, \dots, g^{p-1} = 1$.

XOR i polinomi: U polju $F_2 = \{0, 1\}$, sabiranje je isto što i XOR. Ovo se proširuje na polinome nad F_2 – sabiraju se koeficijenti po modulu 2 (bez prenosa). Npr. $(x^3 + x + 1) + (x^2 + x) = x^3 + x^2 + 1$. Ova aritmetika polinoma se koristi unutar AES-a (polje F_{2^8}).

Inverz u konačnom polju se računa pomoću proširenog Euklidovog algoritma ili pomoću Male Fermaove teoreme: $a^{-1} \equiv a^{p-2} \pmod{p}$.

Difi-Helmanova razmena (usaglasavanje) ključa

Ovde koristimo konačno polje F_q i jedan element $g \in F_q$. Najbolje je da g bude generator multiplikativne grupe $F_q^* (= F_q \setminus \{0\})$, a prihvatljivo je i da bude element velikog reda. **Difi-Helmanova razmena ključa** se zasniva na sledecem:

- Ako znamo $g \in F_q^*$ i $n \in \mathbb{N}$ lako je odrediti g^n
- Ako znamo g i g^n tesko je odrediti n

Algoritam:

1. Primalac i Posiljalac biraju stepen prostog broja $q = p^d$ (približno 200-cifren) i generator $g \in F_q^*$ i objavljuju q i g
2. Posiljalac bira svoj tajni ključ $a_A \in \mathbb{N}$, računa i objavljuje samo g_A^a (tj. javni ključ)
3. Slično, primalac bira tajni ključ $a_B \in \mathbb{N}$, računa i objavljuje samo g_B^a (javni ključ)
4. Posiljalac i primalac oba mogu da izračunaju $K = (g_A^a)_B^a = (g_B^a)_A^a$ i to predstavlja njihov usaglasen javni ključ
5. Prisluskivac zna samo q, g, g_A^a i g_B^a i pomoću toga ne može u razumnom vremenu odrediti K

Da bi mogli brzo da izračunamo $g^n \in F_q^*$ koristimo algoritam **stepenovanja ponovljenim kvadriranjem**.

Stepenovanje kvadriranjem i složenost

Algoritam **stepenovanja ponovljenim kvadriranjem** je sledeći:

1. Redukovati stepen na $n < q-1$ zbog činjenice da je $g^{q-1} = 1$ na osnovu *male Fermaove teoreme*
2. Zapisati n binarno (kao sumu stepena 2^i , gde je $i \in \{0, 1, 2, \dots, r\}$)
3. Izračunati $1, g, g^2, (g^2)^2, (g^2^2)^2, \dots, g^{2^r}$ (svaki je kvadrat prethodnog)
4. g^n je proizvod onih g^{2^i} za koje je $n_i = 1$

U slučaju $q = p$ prost složenost ovog algoritma je $O(r(\log p)^2) = O((\log p)^3)$, dok $g^n = g * g * g \dots * g$ bi trebalo $O(n(\log p)^2)$ operacija. Takođe na osnovu Male Fermaove Teoreme može se izračunati da je složenost $O((\log q)^4)$

Problem diskretnog logaritma u konacnom polju

Def. Neka je G grupa (npr F_q^*) i neka su $a, g \in G$. Najmanji prirodan broj n (ako postoji) takav da je $a = g^n$ zovemo **diskretni logaritam** od a u osnovi g i oznacavamo sa $\log_g(a)$.

Problem: Nemamo formulu da izracunavanje $n = \log_g(a) \in F_q^*$, sto znaci da ne postoji dovoljno brz algoritam koji resava problem diskretnog logaritma u F_q^* tj. algoritam cija je brzina uporediva sa stepenovanjem, obicna pretraga je slozenosti $O(q)$, ali postoje optimizacije koje mogu spustiti ovu slozenost na $O(\sqrt{q}(\log p)^2)$, sto u svakom slucaju nije dovoljno brzo.

Blokovske sifre i AES

Problem: Ana i Boban zeleva da komuniciraju poverljivo putem nebezbednog javnog kanala (npr preko WiFi). Eva koja kontrolise kanal moze da prisluškuje komunikaciju, ali i da menja sadržaj svake poruke. Na koji nacin Ana i Boban mogu da ostvare poverljivu komunikaciju i da pritom otkriju ukoliko je poruka bila izmenjena?

Blok šifre su osnovne kriptografske primitive nad kojima je izgrađena većina modernih šifarskih sistema. Osim što nude rešenje za problem poverljive komunikacije, takođe omogućavaju konstrukciju takozvane autentifikovane enkripcije.

Formalno, blok šifra je šifra (E, D) pri cemu je velicina poruke samim tim i šifrata fiksirana na n bitova. Kazemo da je n velicina bloka. Naglasimo da se, zbog tog uslova, blok šifrom ne mogu direktno šifrovati proizvoljne poruke. Za fiksiran ključ k , funkcija $E_k(m) = E(k, m)$ je permutacija skupa svih bitovskih niski duzine n . Cilj prilikom dizajniranja blok šifre je da se fja E_k ponasa kao random permuracija za svaki ključ k .

Uopsteno, blok šifre se konstruisu iterativnom primenom neke jednostavne invertibilne transformacije koja zavisi od ključa, pri cemu se jedna iteracija naziva runda, a transformacija se naziva funkcija runde. Ključ k se prosiruje u niz podključeva $k_1, \dots, k_r$ (po jedan za svaku rundu) pomocu PGR.

Blokovska šifra je tip simetričnog sistema koji podatke obrađuje u **blokovima** fiksne dužine. Dok protočne šifre rade bit-po-bit, blokovske šifre grupišu simbole (npr. 128 bita kod AES-a) i transformišu ih odjednom. **AES (Advanced Encryption Standard)** je postao standard 2001. godine kao naslednik starijeg DES-a, nudeći veću sigurnost kroz duže ključeve (128, 192 ili 256 bita).

Uprošćeni AES (SAES)

SAES je edukativni model koji koristi 16-bitne blokove i 16-bitni ključ kako bi se lakše razumela struktura pravog AES-a. Koristi **tabelu S** (tzv. **S-box**), cija struktura se opisuje koriscenjem konacnog polja $F_{16} = F_2[x]/(x^4 + x + 1)$ od 16 elemenata. Rec. **ni**bl oznacava cetvorku bita npr. 1011. Niblu $b_0b_1b_2b_3$ moze se pridruziti elemenat $b_0x^3 + b_1x^2 + b_2x + b_3$ polja F_{16} .

Tabela S je bijektivno preslikavanje $S: \{0, 1\}^4 \rightarrow \{0, 1\}^4$ niblova u niblove. Ova funkcija je kompozicija dva preslikavanja.

1. Prvo preslikavanje je inverzija nibla u F_{16} , npr. inverz polinoma $x + 1$ je polinom $x^3 + x^2 + x$, pa komponenta preslikavanja S preslikava 0011 u 1110. Nibl 0000 je izuzetak, on se slika sam u sebe. Tako dobijenom niblu N se pridružuje elemenat $N(y) = b_0y^3 + b_1y^2 + b_2y + b_3$.
2. Druga komponenta preslikavanja S je transformacija nibla $N(y)$ u nibl $a(y)N(y) + b(y)$. Tako se npr. nibl 1110 preslikava u nibl 1011. Prema tome $S(0011) = 1011$.

Algoritam AES ima 16-bitni ključ $k_0k_1 \dots k_{15}$. Od njega se formira niz od 48 bita (tri 16-bitna **potključa**; od tih 48 bita prvih 16 jednaki su originalnom ključu). Ovo proširivanje sa 16 na 48 bita naziva se **proširivanje ključa**. Proces koristi funkcije **RotByte** (rotacija bajtova), **SubByte** (zamena preko S-tabele) i dodavanje konstanti **RCON** koje služe da se razbiju pravilnosti i sličnosti između podključeva.

Algoritam se sastoji od početnog koraka i dve runde transformacija. Operacije unutar rundi su:

- **AddRoundKey (AK):** Sabiranje stanja (podataka) sa potključem runde pomoću bitovne XOR operacije.
- **Nibble Substitution (NS):** Nelinearna zamena svakog nibla vrednošću iz **S-tabele** kako bi se postigla konfuzija.
- **ShiftRow (SR):** Cikličko pomeranje (zamena mesta) niblova u drugoj vrsti matrice stanja.
- **MixColumn (MC):** Množenje kolona matrice stanja fiksnim polinomom radi postizanja **difuzije** (širenja uticaja jednog bita na ostatak bloka),

Dešifrovanje se vrši primenom **inverznih operacija** u obrnutom redosledu. Pošto su operacije poput AddRoundKey i ShiftRow (u AES verziji) same sebi inverzne ili se lako invertuju, proces je veoma sličan šifrovanju. Zahvaljujući izostavljanju MixColumn-a u poslednjoj rundi šifrovanja, moguće je organizovati dešifrovanje tako da ima identičnu strukturu koraka, što olakšava hardversku implementaciju.

Modovi rada blok šifri

Blokovska šifra sama po sebi šifruje samo **jedan blok**. Da bi šifrovali poruku dužu od jednog bloka, koriste se **modovi rada** (operacioni modovi).

ECB (Electronic Codebook)

Najjednostavniji mod: svaki blok se šifruje nezavisno istim ključem.

- Šifrovanje: $C_i = E(K, M_i)$
- Dešifrovanje: $M_i = D(K, C_i)$

Prednost: paralelizacija, otpornost na greške u jednom bloku. **Mana:** isti blokovi otvorenog teksta daju iste blokove šifrata → otkriva obrasce u podacima (poznati primer: ECB Tux pingvin).

⚠ ECB se nikada ne koristi u praksi za podatke koji imaju strukturu.

CBC (Cipher Block Chaining)

Svaki blok se pre šifrovanja XOR-uje sa prethodnim šifrovanim blokom:

- Šifrovanje: $C_i = E(K, C_{i-1} \oplus M_i)$, gde je $C_0 = IV$
- Dešifrovanje: $M_i = D(K, C_i) \oplus C_{i-1}$

IV (inicijalizacioni vektor) je slučajan, ne-tajni broj koji se šalje uz poruku. Obezbeđuje da ista poruka šifrovana dva puta daje različit rezultat.

Prednost: isti blokovi daju različite šifrate, dešifrovanje je paralelizabilno. **Mana:** šifrovanje je sekvencijalno (svaki blok zavisi od prethodnog).

CTR (Counter Mode)

Pretvara blokovsku šifru u protočnu. Šifruje brojač (nonce + redni broj) i XOR-uje sa otvorenim tekstom:

- $K_i = E(K, nonce || i)$
- $C_i = M_i \oplus K_i$

Prednosti: potpuna paralelizacija, ne treba padding, dešifrovanje = šifrovanju. **Mana:** ponovljeni nonce potpuno kompromituje sistem (isto kao kod protočnih šifri).

Problemi sa ECB modom i protočnim šiframa

Problem sa ECB: Obrazci u podacima ostaju vidljivi jer se identični blokovi šifruju u identične šifrate. Primer: šifrovanje bitmap slike u ECB modu ostavlja obrise vidljivim.

Problem sa protočnim šiframa (Two-Time Pad): Ako se isti keystream koristi za dve poruke:

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2$$

Napadač dobija XOR dva otvorena teksta i može statistički rekonstruisati obe poruke.

Meet-in-the-Middle napad (na dvostruki DES)

Zašto ne koristimo prosto 2DES (duplo šifrovanje sa 2 ključa)? $C = E(K_2, E(K_1, M))$

Odgovor: **Meet-in-the-Middle napad** svodi bezbednost dvostrukog DES-a skoro na bezbednost običnog DES-a!

Kako radi:

1. Za poznati par (M, C):
2. Izračunaj $E(K_1, M)$ za svaki mogući $K_1 \rightarrow$ čuvaj u tabeli
3. Izračunaj $D(K_2, C)$ za svaki mogući $K_2 \rightarrow$ traži poklapanje u tabeli

4. Složenost: $2^{56} + 2^{56} = 2^{57}$ umesto očekivanih 2^{112}

Dvostruki DES sa $2 \times 56 = 112$ bita ključa daje samo 2^{57} sigurnosti! Zato se koristi **3DES** (triple DES):
 $C = E(K_1, D(K_2, E(K_1, M)))$

Nedelja 3: Hesiranje

RC4

RC4 je svojevremeno bila jedna od najpopularnijih **protočnih šifara**. Autor ovog algoritma je Ronald Rivest (poznat i kao slovo R u RSA sistemu), a konstruisan je 1987. godine.

Najpre se bira prirodni broj n (obicno se koristi $n=8$). Osnovna komponenta generatora je promenljiva tabela S duzine 2^n , ciji je sadržaj u svakom trenutku neka permutacija brojeva $i=0, 1, \dots, 2^n-1$. U fazi pripreme generatora se na osnovu ključa izracunava pocetni sadržaj niza S tj. neka permutacija skupa $\{0, 1, \dots, 2^n-1\}$. Izracunavanje pocinje tako sto se stavi S_i za $i \in \{0, 1, \dots, 2^n-1\}$. Zatim se od ključa formira drugi niz $K_0, K_1, \dots, K_{2^n-1}$ od 2^n n -torki bita.

```
# priprema generatora
j = 0
for j in range(0, 2^n):
    j = (j + S_i + K_i) % 2^n
    swap(S_i, S_j)

# generisanje blokova niza ključa
i = 0, j = 0
for r in range(0, l)
    i = (i + 1) % 2^n
    j = (j + S_i) % 2^n
    swap(S_i, S_j)
    t = S_i + S_j
    KS_r = S_t # naredna n-torka bita izlaznog niza ključa
```

Heširanje

Heširanje je proces transformacije ulaznih podataka bilo koje velicine u izlaz fiksne duzine, koji se naziva **heš vrednost** koriscenjem neke funkcije za heširanje. Za razliku od enkripcije, heširanje je **jednosmerna funkcija** tj. dizajniraju se tako da bude nemoguće vratiti heš u originalne podatke. Služe nam da obezbedimo da poruku koju smo primili neko usput nije izmenio, osobina poruke da nije dozvela izmene je **integritet**. Svaka heš funkcija treba da ima sledeća svojstva:

- Potrebno je da se njene vrednosti lako i brzo izracunavaju
- Heš funkcija treba da bude jednosmerna funkcija (za zadato y potrebno je da bude jako tesko ili nemoguće oredjivanje x takvog da $H(x) = y$)

- Za zadato x potrebno je da bude jako tesko odredjivanje drugog x' t.d. $H(x') = H(x)$. Ova osobina se naziva **osnovna otpornost na koliziju**

Ako je pored toga ispunjen zahtev da je tesko pronaci bilo koji par x, x' takav da je $H(x) = H(x')$, onda se za H kaze da ima **jaku otpornost za koliziju**.

Da se napravi algoritam za heširanje obicno se polazi of funkcije f koja blokove od $m + t$ bita preslikava u blokove od t bita, gde su m i t veliki, a f ima sve tri navedene osobine. Ulaz ovog algoritma je poruka, a izlaz njena heš vrednost.

Funkcija f moze se iskoristiti za dobijanje heš funkcije. Pretpostavimo da je poruka razbijena u m -bitne blokove $M_1, M_2, \dots, M_k$. Ako duzina poruke nije deljiva sa m , onda se poslednji blok dopunjuje do duzine m . Zadat je unapred dogovoren blok od t bita (inicijalizovani vektor IV).

MD5

Jedan od najpopularnijih hes algoritama je MD5. On je efikasniji od malo pre opisanog algoritma koji koristi AES. On se zasniva na funkciji f koja preslikava blok od 512 bita u blok od 128 bita. Neka je M blok od 512 bita. U ovom kontekstu zvacemo blok od 32 bita rec, dakle M se sastoji od 16 reci - $X[0], X[1], \dots, X[15]$.

Tokom izvršavanja stalno se azuriraju stanja cetiri 32-bitna registara. Na pocetku je $A = A_0, B = B_0, C = C_0, D = D_0$. Definiseemo cetiri funkcije:

- $F(X, Y, Z) = XY \vee !XZ$
- $G(X, Y, Z) = XZ \vee Y!Z$
- $H(X, Y, Z) = X \oplus Y \oplus Z$
- $I(X, Y, Z) = Y \oplus (X \vee !Z)$

Postoje i 64 konstante $T[1], \dots, T[64]$. Neka je i broj radijana, tada je $|\sin(i)|$ realni broj izmedju 0 i 1 i $T[i]$ je prvih 32 bita posle decimalne tacke.

Sam algoritam radi tako sto se u unutar petlje, jedan od 4 registara (tj. vrednost unutar njega, uzmimo da je to ovde A) azurira na sledeci nacin:

1. **Nelinearna transformacija:** Primenjuje se neka od funkcija F, G, H na preostale vrednost (B, C, D)
2. **Modularno sabiranje:** Rezultat funkcije se sabira sa promenljivom koja se azurira (A), jednim podblokom poruke i nekom konstantom iz tabele
3. **Ciklicno pomeranje (<<):** Dobijeni zbir se ciklicno pomera ulevo za odredjen broj bita koji se razlikuje za svaku operaciju unutar runde kako bi se osiguralo maksimalno mesanje
4. **Finalno sabiranje:** Na kraju se dobijeni rezultat sabira sa vrednoscu B , a zatim sve 4 rotiraju mesta, $A \rightarrow D \rightarrow C \rightarrow B \rightarrow A$.

Nakon što se završe sve 64 operacije za jedan blok, trenutne vrednosti A, B, C, D se sabiraju sa njihovim vrednostima koje su imale pre početka obrade tog bloka. Ovaj proces se ponavlja za svaki

blok od 512 bita u poruci. Kada se obrade svi blokovi, finalne vrednosti varijabli A, B, C, D se spajaju u niz. Svaka varijabla je dugačka 32 bita, što ukupno daje **128 bita** ($32 \text{ bita} \times 4$). Taj binarni niz se konvertuje u heksadecimalni format, čime se dobija MD5 potpis od 32 karaktera.

MAC

MAC (Message Authentication Code), ili **autentikacioni kod poruke**, predstavlja kriptografski alat koji se koristi za obezbeđivanje **integriteta** i **autentikacije** poruke. To je praktično **hes funkcija koja koristi tajni ključ**. Dok obične hes funkcije (poput MD5) zavise samo od same poruke, MAC zavisi i od poruke i od tajnog ključa koji dele pošiljalac i primalac. Na ovaj način postizemo:

- **Integritet:** Osigurava da poruka nije promenjena tokom prenosa.
- **Autentikacija:** Potvrđuje primaocu da je poruku zaista poslao onaj ko poseduje tajni ključ.

Kako proces funkcioniše u praksi:

1. **Dogovor:** Posiljalac i primalac se unapred dogovore o zajedničkom tajnom ključu k
2. **Slanje:** Posiljalac šalje primaocu poruku (može biti šifrovana ili u običnom tekstu) i uz nju prilaže izračunatu MAC vrednost te poruke koristeći tajni ključ k
3. **Provera:** Primalac prima poruku i MAC. On samostalno izračunava MAC primljene poruke koristeći isti onaj ključ k koji deli sa posiljaocom.
4. **Verifikacija:** Ako se primaocov izračunati MAC slaže sa onim koji je Alisa poslala, on je siguran da:
 - Poruka nije promenjena (integritet).
 - Poruku je poslalo onaj ko je trebao, jer niko drugi ne zna njihov tajni ključ (autentikacija).

Zašto je MAC neophodan? Bez MAC-a, napadač bi mogao da presretne šifrat i namerno promeni neke njegove delove. Čak i ako napadac ne može da pročita poruku jer ne zna ključ za dešifrovanje, ona može da je pokvari. Primalac bi nakon dešifrovanja dobio besmislenu poruku, ali ne bi znao da li je greška nastala u prenosu ili je neko namerno manipulisao podacima. **Napadac ne može da kreira ispravan MAC** za izmenjenu poruku jer ne poseduje tajni ključ.

Primeri i primena:

- **CBC-MAC:** Koristi blokovsku šifru poput AES-a u CBC režimu. Rezultat šifrovanja poslednjeg bloka (uz korišćenje tajnog ključa umesto inicijalizacionog vektora) služi kao MAC.
- **HMAC (varijante):** Kombinovanje hes funkcija i ključa, npr. izrazom $H(K \parallel H(K \parallel M))$, gde je K ključ, a M poruka.
- **Primena u TLS protokolu:** Kod HTTPS-a (TLS protokol), MAC se često koristi za svaku poruku u okviru sesije kako bi se osigurala bezbednost podataka. Na primer, može se koristiti SHA1 algoritam gde tajni ključ služi kao inicijalizacioni parametar.

Digitalni potpis

Digitalni potpis je kriptografski mehanizam koji se koristi za obezbeđivanje **autentikacije**, **integriteta** i **neporecivosti** elektronskih poruka ili dokumenata. Dok digitalni potpis povezuje samu poruku sa javnim ključem pošiljaoca, prateći sertifikat je taj koji povezuje taj javni ključ sa konkretnom osobom.

Digitalni potpisi se primarno zasnivaju na **kriptografiji sa javnim ključem** (asimetričnoj kriptografiji) i **jednosmernim funkcijama**.

Kako radi:

1. Pošiljalac izračuna heš poruke: $h = H(M)$
2. Šifruje heš svojim **privatnim ključem**: $sig = Sign(K_{priv}, h)$
3. Šalje: (M, sig)

Verifikacija (primalac):

1. Izračuna heš primljene poruke: $h' = H(M)$
2. Dekriptuje potpis **javnim ključem** pošiljaoca: $h = Verify(K_{pub}, sig)$
3. Proveri: $h == h' \rightarrow$ potpis je validan

Razlika od MAC-a: MAC je simetričan (ko god ima ključ može i da generiše i da verifikuje) – ne daje **neporecivost**. Digitalni potpis je asimetričan: samo vlasnik privatnog ključa može da potpiše, ali svako sa javnim ključem može da verifikuje.

RSA

RSA (Rivest-Shamir-Adleman) je najpoznatiji algoritam asimetrične kriptografije. Bezbednost se zasniva na težini **faktorizacije velikih brojeva**.

Generisanje ključeva:

1. Izaberi dva velika prosta broja p i q
2. Izračunaj $n = p \cdot q$
3. Izračunaj Ojlerovu funkciju $\phi(n) = (p - 1)(q - 1)$
4. Izaberi e takvo da $1 < e < \phi(n)$ i $NZD(e, \phi(n)) = 1$
5. Izračunaj d tako da $e \cdot d \equiv 1 \pmod{\phi(n)}$

Javni ključ: (n, e) | **Privatni ključ:** d

Šifrovanje: $C = M^e \pmod{n}$

Dešifrovanje: $M = C^d \pmod{n}$

Zašto radi? Ojlerova teorema: $M^{e \cdot d} \equiv M^{1+k\phi(n)} \equiv M \pmod{n}$

Primer (p=11, q=5): $n = 55, \phi(n) = 40, e = 3, d = 27$. Poruka $M = 7$: $C = 7^3 \bmod 55 = 13$.
Dešifrovanje: $M = 13^{27} \bmod 55 = 7$.

Sigurnost: Ako napadač faktorizuje n , može izračunati $\phi(n)$, pa d . Preporučene dužine ključa: minimalno 2048 bita.

Rodendanski paradoks

Objasnjava zašto heš mora biti dovoljno dugačak.

Paradoks: U grupi od samo 23 osobe, verovatnoća da dve imaju isti rodendan je $> 50\%$.

Uticaj na kriptografiju: Za heš od n bita:

- Brute-force za jednosmernost (naći inverz): $\sim 2^n$ pokušaja
- Brute-force za koliziju (naći bilo koji par): $\sim 2^{n/2}$ pokušaja!

Zato MD5 (128b $\rightarrow$ kolizija za 2^{64}) i SHA-1 (160b $\rightarrow 2^{80}$) **nisu sigurni**. SHA-256 zahteva 2^{128} pokušaja za koliziju – sigurno.

Obavezujuća šema (Commitment Scheme)

Mehanizam koji omogućava da se osoba **obaveže na izbor** bez da ga otkrije, a da posle ne može da ga promeni.

Problem: Bacanje novčića preko telefona – ko god baca može da laže.

Rešenje:

1. Alisa baca novčić, dobija ishod I (glava/pismo)
2. Alisa bira random **salt** S
3. Alisa šalje Bobanu: $\text{commitment} = H(I||S)$
 - Boban ne može iz heša da sazna I (jednosmernost)
 - Alisa ne može da promeni I jer bi se promenio heš (otpornost na kolizije)
4. Boban kaže šta se dešava u kom slučaju
5. Alisa otkriva: I i S
6. Boban proverava: $H(I||S) == \text{commitment}?$

Zašto salt? Bez salta, Boban bi mogao da proba $H(\text{"glava"})$ i $H(\text{"pismo"})$ i sazna ishod.

HMAC (Hash-based Message Authentication Code)

Obezbeđuje **integritet** i **autentičnost** poruke koristeći heš funkciju i deljeni tajni ključ.

Naivni pristup (pogrešan): $MAC = H(K||M)$ – ranjiv na **length extension napad** kod Merkle-Damgård heš funkcija (MD5, SHA-1, SHA-2). Napadač može iz $H(K||M)$ da izračuna $H(K||M||padding||M')$ bez poznavanja ključa!

Ispravan HMAC:

$$HMAC(K, M) = H((K \oplus opad) || H((K \oplus ipad) || M))$$

Dvostruko heširanje sa različitim padding-ima ($ipad=0x36$, $opad=0x5c$) sprečava length extension napad.

Merkle stablo

Hijerarhijska struktura heševa za **efikasnu verifikaciju integriteta** velikih skupova podataka.

Princip:

- **Listovi:** heševi pojedinačnih podataka
- **Unutrašnji čvorovi:** heš konkatencije svoje dece
- **Koren (Merkle Root):** jedan heš koji predstavlja sve podatke

Ključno svojstvo: Promena bilo kog podatka u listu menja koren stabla.

Efikasna verifikacija (Merkle proof): Da dokažeš da je podatak u skupu od N elemenata, treba ti samo $O(\log N)$ heševa umesto svih N .

Primene: Git (commit = heš korena), Blockchain (svaki blok sadrži Merkle Root transakcija), efikasno poređenje velikih skupova fajlova.

KDF (Key Derivation Function)

Funkcija koja od **slabe lozinke** pravi **jak kriptografski ključ**.

Zašto je potrebna? Korisničke lozinke su kratke i predvidive – loš materijal za ključ. KDF dodaje:

- **Sporost:** Probanje jedne lozinke traje dugo → brute-force je skuplji
- **Salt:** Ista lozinka + različit salt → potpuno različit izlaz (sprečava rainbow tabele)
- **Memorijska zahtevnost:** Teža paralelizacija na GPU/ASIC

Čuvanje lozinki u bazi:

- Registracija: $salt = random()$, $hash = KDF(lozinka, salt, parametri)$, čuvaj $(salt, hash)$
- Login: izračunaj $KDF(uneta_lozinka, salt)$ i uporedi sa čuvanim hešom

Poznate KDF: PBKDF2 (iterativno heširanje), bcrypt (Blowfish), scrypt (memorijski zahtevan), **Argon2** (preporučen danas – konfigurabilan CPU + memorija).

Nedelja 4: Asimetrična kriptografija – ElGamal, Eliptičke krive, Sertifikati

Mesi-Omura protokol (idejno)

Analogija sa katancima – ilustruje da je moguća bezbedna komunikacija **bez zajedničkog ključa**:

1. Alisa stavlja poruku u kutiju, zaključava **svojim** katancem → šalje Bobanu
2. Boban NE MOŽE otvoriti, ali stavlja i **svoj** katanac → šalje nazad Alisi
3. Alisa skida **svoj** katanac → šalje Bobanu
4. Boban skida **svoj** katanac → čita poruku

Niko osim Alise i Bobana nije mogao da čita poruku, a nikad nisu razmenili ključ! U praksi se implementira korišćenjem komutativnog šifrovanja (gde je redosled primene ključeva nebitan).

Diffie-Hellman razmena ključeva (pogled sa vežbi)

Nije sistem za šifrovanje, već protokol kojim dve strane **dogovaraju zajednički tajni ključ** preko nesigurnog kanala. Zasniva se na **problemu diskretnog logaritma**.

Protokol:

1. Javni parametri: veliki prost p , generator g
2. Alisa bira privatno a , računa $A = g^a \bmod p$ i šalje Bobanu
3. Boban bira privatno b , računa $B = g^b \bmod p$ i šalje Alisi
4. Alisa računa: $S = B^a \bmod p = g^{ab} \bmod p$
5. Boban računa: $S = A^b \bmod p = g^{ab} \bmod p$ – **isto!**

Šta napadač vidi? g, p, A, B – ali da bi izračunao S , morao bi da nađe a ili b (problem diskretnog logaritma).

Problem: DH sam po sebi ne štiti od **Man-in-the-Middle** napada. Rešenje: sertifikati.

ElGamal algoritam za šifrovanje

Asimetrični kriptosistem zasnovan na problemu diskretnog logaritma.

Javni podaci: p (prost broj), g (generator), $y = g^x \bmod p$ (javni ključ primaoca)

Privatni ključ primaoca: x

Šifrovanje (pošiljalac šalje poruku M):

1. Bira slučajan ključ sesije k
2. Računa $c_1 = g^k \bmod p$
3. Računa $c_2 = M \cdot y^k \bmod p$
4. Šalje par (c_1, c_2)

Dešifrovanje (primalac koristi privatni ključ x):

1. Iz c_1 računa $S = c_1^x = (g^k)^x = g^{kx} \bmod p$
2. Računa inverz: $S^{-1} \bmod p$
3. Dobija poruku: $M = c_2 \cdot S^{-1} \bmod p$

Bitno: Random k se **mora menjati** za svako šifrovanje. Ponavljanje k potpuno kompromituje sistem. Šifrat je **dva puta duži** od poruke.

Elipsičke krive (ECC)

Elipsička kriva je kriva data jednačinom: $y^2 = x^3 + ax + b$ (Weierstrass forma)

To nije elipsa – naziv dolazi od elipsičkih integrala. Kriva je simetrična oko x-ose i ima posebnu "tačku u beskonačnosti" $\mathcal{O}$ koja služi kao neutralni element.

Sabiranje tačaka

Nad tačkama krive definišemo operaciju "+":

$P + Q$ (**različite tačke**): Pravuču pravu kroz P i Q , ona seče krivu u trećoj tački R' . Refleksija R' preko x-ose daje $P + Q$.

$2P$ (**udvostručavanje**): Tangenta u P seče krivu u R' , refleksija daje $2P$.

Specijalni slučajevi: $P + \mathcal{O} = P$, $P + (-P) = \mathcal{O}$

Ovim dobijamo **grupu** – skup tačaka sa asocijativnom operacijom, neutralnim elementom i inverzima.

Analogija sa klasičnim DLP

Konačno polje F_q^*	Elipsička kriva E
Množenje $a \cdot b$	Sabiranje tačaka $P + Q$
Stepenovanje g^n	Skalarno množenje nG
Neutralni element 1	Tačka $\mathcal{O}$

Konačno polje F_q^*	Eliptička kriva E
Generator g	Generator-tačka G

Krive nad konačnim poljem

Za kriptografiju radimo nad F_p (gde je p veliki prost broj):

- Tačke su parovi (x, y) sa $x, y \in \{0, \dots, p-1\}$ koji zadovoljavaju $y^2 \equiv x^3 + ax + b \pmod{p}$
- Skup tačaka $E(F_p)$ je konačan ($\sim p + 1$ tačaka)
- Sve operacije se računaju po modulu p

Problem diskretnog logaritma sa eliptičkim krivama (ECDLP)

- **Lako:** dato G i $n \rightarrow$ izračunati nG (algoritmom "double-and-add", $O(\log n)$)
- **Teško:** dato G i $Q = nG \rightarrow$ naći n

Bolji napadi na ECDLP su **eksponencijalni** (za razliku od subeksponencijalnih napada na klasičan DLP), pa za istu sigurnost možemo koristiti **mного kraće ključeve**:

ECC ključ	RSA ključ	Sigurnost
256 bita	3072 bita	~ 128 bita
384 bita	7680 bita	~ 192 bita

ElGamal sa eliptičkim krivama

Isti princip kao običan ElGamal, samo se menja: množenje $\rightarrow$ sabiranje, stepenovanje $\rightarrow$ skalarno množenje.

Ključ primaoca: privatni x , javni xG (tačka na krivoj)

Šifrovanje poruke (tačka Q):

- Bira slučajno k
- Šalje par: $(kG, Q + k(xG))$

Dešifrovanje:

- Iz kG i privatnog x računa $x(kG) = (xk)G$
- Oduzima: $(Q + k(xG)) - (xk)G = Q$

Digitalni potpisi (dopuna)

Proces potpisivanja:

1. Izračunaj heš poruke: $h = H(M)$
2. Potpiši hešom privatnim ključem: $sig = Sign(K_{priv}, h)$
3. Pošalji: (M, sig)

Zašto heširati pre potpisivanja? RSA potpis je spor za velike podatke. Heš svodi poruku na fiksnu dužinu (npr. 256b) pa se potpisuje samo heš.

Sign-then-Encrypt vs Encrypt-then-Sign:

- **Sign-then-Encrypt:** Potpis + poruka $\rightarrow$ šifruj sve. Smatra se boljim jer potpis štiti originalni tekst.
- **Encrypt-then-Sign:** Šifruj poruku $\rightarrow$ potpiši šifrat.

Sertifikati i CA (Certificate Authority)

Problem: DH i svi asimetrični sistemi bez autentikacije su podložni **Man-in-the-Middle** napadu – napadač se ubaci između i uradi DH sa obe strane.

Sertifikat je digitalni dokument koji vezuje **javni ključ za identitet**. Sadrži:

- Javni ključ vlasnika
- Identitet (domen, npr. google.com)
- Izda vač (CA koji je potpisao sertifikat)
- Period važenja
- **Potpis CA** – ovo garantuje autentičnost

Lanac poverenja (Chain of Trust):

- Root CA (čiji su ključevi ugrađeni u browser/OS)
 - Intermediate CA (potpisan od Root CA)
 - Server sertifikat (potpisan od Intermediate CA)

Klijent se penje lancem dok ne dođe do Root CA kome veruje.

Opozivanje sertifikata: CRL (lista opozvanih), OCSP (online provera u realnom vremenu), OCSP Stapling (server kešira odgovor CA i šalje ga klijentu).

Vremenski pečati (TSA)

Dokaz da je dokument postojao u određenom trenutku:

1. Izračunaj heš dokumenta: $h = H(dokument)$
2. Pošalji h ka TSA (Time Stamping Authority)
3. TSA nadoveže tačno vreme i potpiše: $potpis = Sign(K_{TSA}, h || vreme)$
4. Dobiješ vremenski pečat: $(h, vreme, potpis)$

Alternativa: **blockchain** – heš se upiše u transakciju čiji je timestamp neopoziv.

Onion šifrovanje (Tor mreža)

Tehnika za **anonimnu komunikaciju** – poruka se šifrjuje u više slojeva:

1. Alisa šifrjuje poruku u 3 sloja: $C = E(K_1, E(K_2, E(K_3, M)))$

2. Svaki čvor skida jedan sloj:

- N_1 : skida spoljašnji sloj, prosleđuje N_2
- N_2 : skida drugi sloj, prosleđuje N_3
- N_3 : skida poslednji sloj, šalje serveru M

3. Nijedan čvor ne zna i pošiljaoca i primaoca:

- N_1 zna: Alisa $\rightarrow N_2$ (ne zna krajnje odredište)
- N_2 zna: $N_1 \rightarrow N_3$ (ne zna ni pošiljaoca ni primaoca)
- N_3 zna: $N_2 \rightarrow$ Server (ne zna originalnog pošiljaoca)

Garlic šifrovanje (I2P mreža)

Slično onion-u, ali sa grupnim slanjem:

- Više poruka se kombinuje u **jedan paket** (kao čenovi belog luka)
- Svaki čvor dešifrjuje samo deo koji mu je namenjen
- Jedan garlic paket sadrži poruke za **različite primaoce** $\rightarrow$ teže je analizirati saobraćaj
- Komunikacija je uglavnom unutar same I2P mreže